

Analysis: Closest Pick

Test Set 1

In Test Set 1, the limits are small enough that we can try every one of the K^2 possible pairs of integers to pick. For each of those, we can calculate the probability of winning with each possible draw. There are K possible values for c . If for each one, we check the distance to each P_i and to our own two picks, we end up with an overall running time of $O(K^3 \cdot N)$. This is fast enough to pass.

There are several possible optimizations. A simple one is to search for the "closest picked integer" using a sorted structure, reducing that phase to $O(\log N)$ and the overall running time to $O(K^2 \cdot N \log N)$. It is also possible to search for the distance from each c to each P_i only once, independently of our choices, which reduces the overall running time further to $O(K^2)$.

Test Set 2

For Test Set 2, we can optimize all phases from the previous solution. If every integer is already picked at least once, then the answer is always 0 as the samples demonstrate. Otherwise, we can divide the non-purchased integers from $1, 2, \dots, K$ into intervals between P_i s. For example, if the P_i values are 3, 4, 8, 3 and $K = 8$, the intervals would be $[1, 2]$, $[5, 7]$.

If we make only one of our picks within one of those intervals, we can get closest to all of it if the interval contains 1 or K , by picking ours next to the only delimiting P_i (picking 2 from $[1, 2]$ in the example above). If the interval is surrounded by two P_i s, we can get closest to at most half of the integers in the interval (rounding up), which is achievable by picking either end (picking either 5 or 7 from $[5, 7]$). If we make both of our picks within one of those intervals, we can get closest to all integers in the interval by picking both ends (5 and 7 from $[5, 7]$).

This shows that only integers that are next to an already purchased integer are worth picking for ourselves. At this point, we can go back to our last Test Set 1 solution and do the same but restricting our picks to integers next to a P_i , which makes the K s in the running time be $O(N)$, making it fast enough with the right implementation.

A few extra reasoning steps lead to a much more efficient solution. There are only $O(N)$ ways to make both picks within the same interval, and we can check each of them. If we choose two different intervals, it is always optimal to choose the two most valuable ones, so we do not need to check each combination separately. This leads to a single additional case that requires $O(N)$ time to compute to find the top two intervals. All in all, this solution takes only linear time (after sorting the array to find the intervals). This solution requires some care about corner cases, specifically the cases when no or just one integer from $[1, K]$ is not a P_i , and the case of the intervals list containing a single interval.

Analysis: Double or NOTing

Test Set 1

Let's construct a graph where the nodes are numbers and the edges are the operations described in the statement. The problem asks us to find the shortest path from node **S** to node **E** in this graph. The graph is infinite, but we can inspect only a limited set of nodes to find the answer.

When thinking about binary representations, the double operation can be described as "add a trailing zero". Note that it is never optimal to add more zeroes than we have bits in **E**: because the NOT operation can only drop leading digits, any additional zeroes will ultimately have to be dropped anyway.

We can use this fact to limit the maximal length to the sum of the lengths of **S** and **E**, and then use a [Breadth-first search](#) algorithm to find the shortest path.

Test Set 2

Let's define a *bit group* as a maximally long group of consecutive zeroes or consecutive ones (that is, a group of all bits b that does not have a b adjacent to it). Let's assume that there are K bit groups in **S** and L bit groups in **E**.

The double operation appends a bit to the right end of the binary string, and the NOT operation complements and possibly drops bits from the left end. Therefore, the result of applying operations to **S** comes from some (possibly empty) suffix of **S** (possibly complemented) and then some extra bits that were created using double. Let us call the bits that come from a bit in **S** *reused*.

First, let's consider the case when we do not reuse any bits from **S**. In such a case we need to construct **E** naively using only the bits added by double operations and removing all the bits that came from **S** originally. We can do this by adding zeroes with double operations and applying a NOT operation whenever we need to start a new bit group. We also need to apply additional NOT operations to eliminate all bits that came from **S** originally. If we end up with **E**, then let's count this as a possible answer. For example, to get from 101_2 to 11100_2 we would need to do perform the following sequence of operations (we use a space to separate bits originated from **S** from the extra bits added with double operations):

Operation	Result
double (to add an extra bit)	101 0
double (to add an extra bit)	101 00
double (to add an extra bit)	101 000
NOT (because the length of first bit group in E is 3, and we have added 3 zeros already, now we need to start a new bit group)	10 111
double (to add an extra bit)	10 1110
double (to add an extra bit)	10 11100
NOT (because we already have the desired E as suffix, now we just need to remove extra bits from S)	1 00011
NOT (to remove the leftover bit of S)	11100

If S ends in 0, we need to perform an extra NOT operation at the beginning so that our first double operation creates a brand new bit group. After we finish removing all bits from S , we might need another NOT operation, if the removal process left us with the complement of E .

Now, let's try to reuse some bits from S .

The NOT operation removes one bit group from the left end of the binary representation and complements the rest. It is never optimal to apply a NOT operation more than $K + 1$ times, as in such a case we will either be looping between 0 and 1, or will be removing bits which were added with the double operation (which we could have just not added in the first place if they were not useful).

Let's assume that the answer will have NOT operations applied exactly X times. Let's apply X NOT operations to S to obtain S' . If S' is not a prefix of E , then it is not possible to get to E from S using X NOT operations, as the double operation will not change the prefix of the string.

If S' is a prefix of E , let's see whether we can construct the suffix. We will need exactly $Y = \text{len}(E) - \text{len}(S')$ double operations. Let's say there are M bit groups in the suffix we need to create, then we will also need to apply M or $M + 1$ NOT operations, depending on the parity of M and the first bit of the prefix. Let's denote the number as Z . If Z is greater than X , then this case does not work, as we would apply more than X NOT operations, violating the assumption. Otherwise, the answer is $X + Y$, as even if X is greater than Z , extra NOT operations won't affect the suffix (we can perform them before adding any extra bits with a double operation).

We can now iterate through all possible values of X from 0 to $K + 1$, inclusive, and choose the minimum answer between all of those and the non-reuse case we considered first. If none of the cases work, we output IMPOSSIBLE.

Analysis: Roaring Years

Test Set 1

A straightforward approach

Notice that 1234567 is a roaring year that happens after every possible input. That means that if we simply check every year after Y to see if it is roaring or not, we need to check no more than 1234567 years per case. It is a lot less in practice, but that is also harder to prove.

To check whether a year y is roaring, we can check for each prefix p of y whether "completing it" by appending $p + 1$, $p + 2$, etc., would work. Since only prefixes that are at most half the length of y can work, this is a small number of things to check. If we are unconvinced about the running time, we can simply run a check of roaringness for every integer up to 1234567 once initially and remember it, after which each test case is simply looking for the next "true" in a rather small table, making the running time mostly independent of the actual input, so a test run can help convince us that it will run in time without risking penalty with a real submission.

An approach with a more clear time complexity

Another way to go is to first use the observation at the end of the solution above: the length of the first number in the concatenation is small — at most 3 digits. We can try each of the 999 possibilities and concatenate numbers to it until it goes above the 1234567 threshold. Notice that this is at most 6. This builds a set of all possible roaring years that matter in very little time. After that, we can use linear search in this very small list (less than 6000 elements) to find the smallest roaring year that is larger than Y for each test case. Of course, we could use [binary search](#) to make each test case run even faster, but that is not required.

Test Set 2

None of the approaches above work for Test Set 2. The gap between two consecutive roaring years can easily be large (for example, you can prove as an exercise that there are no other roaring years between 100000000100000001 and 100000001100000002) and checking an individual one, while quick, takes a bit more time than before. For the second approach, there are now $10^9 - 1$ candidates for the first number in the concatenation, and the amount of concatenated numbers can also be much larger for some of those, making the full set of roaring years too big to even fit in memory, not to mention the computation time.

A case-based approach

The last sentence of the previous paragraph hints at an idea. While the full set of roaring years in range is too big, the full set of roaring years that are the concatenation of 3 or more numbers is not: for that case, the maximum possible starting number has 6 digits. We can use the second Test Set 2 solution with this change to solve for those. We still need to check roaring years that are the concatenation of exactly two consecutive numbers. Notice that $f(x)$ = the concatenation of x and $x + 1$ is an increasing function. Therefore, we can use the [bisection method](#) to efficiently find the minimum of that subset of roaring years that is above Y . After we found the best candidate from each subset, we simply return the smallest of them.

A general approach

We can also generalize the second case of our first approach for Test Set 2. The family of functions $f_n(x) = \text{concatenation of } x, x + 1, \dots, x + n - 1$ are all increasing. We can therefore use bisection to find the best candidate for each possible amount of consecutive numbers n , and then answer the best of those. Since n is at most $\log \mathbf{Y}$ this yields a method that runs in $O(\log^2 \mathbf{Y})$ time, which means it can work for really large bounds.